

FUNDAMENTOS DE JSX E COMPONENTES

Sumário:

1. Introdução ao JSX.....	2
1.1 Sintaxe Básica.....	2
1.2 Como Funciona o JavaScript Embutido.....	2
1.3 Como o JSX é Interpretado.....	3
2. Introdução aos elementos e componentes React.....	4
2.1 Elementos React.....	4
2.2 Componentes React.....	4
3. Funcionamento de Props e Children no React.....	7
3.1 O Que São Props?.....	7
3.2 Passagem de Funções via Props.....	9
3.3 Children: Composição de Componentes.....	10
3.4 Diferenças entre Props e Children.....	13
3.5 Boas Práticas no uso de Props e Children.....	13
4. Renderização Condicional e Renderização de Listas em React.....	16
4.1 Renderização Condicional.....	16
4.2 Renderização de Listas.....	19
4.3 Combinação de Renderização Condicional e Listas.....	22
Referências.....	24

1. Introdução ao JSX

JSX (JavaScript XML) é uma extensão de sintaxe do JavaScript utilizada no React para descrever a interface do usuário de forma simples e intuitiva. Embora se pareça com HTML, o JSX permite que você escreva HTML diretamente dentro de arquivos JavaScript, combinando o poder do JavaScript com a simplicidade da marcação.

1.1 Sintaxe Básica

No JSX, o código HTML é escrito dentro do JavaScript, mas com algumas diferenças sutis. Cada elemento JSX precisa estar envolto em um único elemento pai, e algumas palavras reservadas do HTML (como **class** e **for**) são substituídas por **className** e **htmlFor** respectivamente.

Veja um exemplo básico de JSX no quadro abaixo:

```
const elemento = <h1>Olá, Mundo!</h1>;
```

1.2 Como Funciona o JavaScript Embutido

Uma das características do JSX é a possibilidade de **embutir expressões JavaScript** dentro da marcação HTML. Para isso, basta envolver o código JavaScript com chaves { } dentro do JSX.

Veja um exemplo de JavaScript embutido no quadro abaixo:

```
const nome = "João";  
const elemento = <h1>Olá, {nome}!</h1>;
```

Neste exemplo, a variável **nome** é avaliada e seu valor é inserido no HTML.

Você também pode usar expressões mais complexas dentro das chaves, como funções e operações lógicas, tal como demonstrado no quadro abaixo:

```
const saudacao = (hora >= 12) ? 'Boa tarde' : 'Bom dia';  
const elemento = <h1>{saudacao}, {nome}!</h1>;
```

1.3 Como o JSX é Interpretado

Embora o JSX pareça uma mistura de HTML e JavaScript, ele não é compreendido diretamente pelos navegadores. O React utiliza um transpilador, tal como Babel ou Esbuild, para converter o código escrito em JSX em chamadas de funções JavaScript equivalentes.

Por exemplo, o código JSX demonstrado no quadro abaixo:

```
const elemento = <h1>Olá, Mundo!</h1>;
```

Será convertido para:

```
const elemento = React.createElement('h1', null, 'Olá, Mundo!');
```

Esse processo transforma o JSX em código JavaScript padrão, que é então interpretado pelo navegador para gerar os elementos da interface.

Neste contexto, o JSX simplifica o desenvolvimento no React ao permitir que HTML e JavaScript coexistam no mesmo arquivo. A capacidade de embutir expressões JavaScript diretamente na marcação facilita a criação de interfaces dinâmicas, enquanto o React se encarrega de converter o JSX em chamadas JavaScript para manipular o DOM eficientemente.

2. Introdução aos elementos e componentes React

Em React, os conceitos de elementos e componentes são fundamentais na construção de interfaces de usuário. Vamos analisar cada um destes conceitos no decorrer desta seção.

2.1 Elementos React

Um elemento React é o menor bloco de construção de uma aplicação React. É uma representação simples de um pedaço de UI (interface do usuário), como um botão, um título, ou uma caixa de texto. Elementos React são objetos imutáveis que descrevem o que você quer ver na tela.

Veja um exemplo de elemento React no quadro abaixo:

```
const element = <h1>Hello, world!</h1>;
```

Neste exemplo, `element` é um elemento React que representa um cabeçalho `<h1>` com o texto **"Hello, world!"**. Esse elemento pode ser renderizado na tela com **ReactDOM.render()**.

Eles representam um único nó na árvore do DOM virtual. Ainda, uma vez criado, um elemento React não pode ser alterado. Se você precisar modificar a UI, você cria um novo elemento com as alterações e o React atualiza a UI de forma eficiente.

OBS: Normalmente, os elementos React não são utilizados diretamente; em vez disso, são gerados e retornados por componentes.

2.2 Componentes React

Um componente React é uma **função** ou **classe** que aceita entradas (chamadas **"props"**) e retorna um ou mais elementos React que descrevem como uma parte da interface deve aparecer. Componentes são mais poderosos do que elementos porque permitem reutilização, composição, e encapsulamento de lógica. Eles podem manter o estado e gerenciar o ciclo de vida da interface.

Neste contexto, em React, há duas formas principais de definir componentes: **Componentes Funcionais** e **Componentes de Classe**. Cada um tem suas próprias características, vantagens e diferenças. Vamos explorar essas distinções, os cenários de uso e as boas práticas associadas a cada tipo.

2.2.1 Componentes Funcionais

Os **componentes funcionais** são definidos como funções JavaScript que retornam elementos React (geralmente representados em JSX). Eles são uma forma mais simples e moderna de criar componentes em React.

Originalmente, componentes funcionais não tinham capacidade de gerenciar seu próprio estado ou ciclo de vida, mas isso mudou com a introdução dos [Hooks](#) no React 16.8. Entretanto, falaremos sobre os Hooks mais adiante neste curso.

Veja no quadro abaixo um exemplo de componente funcional com React:

```
function Saudacao(props) {  
  return <h1>Olá, {props.name}!</h1>;  
}
```

Neste exemplo, **Saudacao** é um componente funcional que recebe **props** e retorna um elemento React (**<h1>**). Este argumento **props** é um objeto que contém todas as propriedades passadas para o componente. Essas propriedades são usadas para personalizar o comportamento ou a aparência do componente. A função **Saudacao** retorna um elemento React, que será renderizado no navegador. A estrutura **<h1>...</h1>** é um exemplo de JSX (JavaScript XML), que é uma extensão de sintaxe usada pelo React que permite escrever HTML dentro do código JavaScript.

Para utilizar este componente, seria necessário chamá-lo em outra parte de nossa aplicação. No quadro abaixo mostra como o componente **Saudacao** poderia ser usado:

```
<Saudacao name="João" />
```

Neste exemplo, quando você usa **<Saudacao name="João" />**, você está criando uma instância do componente **Saudacao** e passando "João" como valor para a propriedade **name**. Logo, dentro do componente **Saudacao**, **props.name** terá o valor "João", e o componente renderizará o elemento **<h1>Olá, João!</h1>**.

2.2.2 Componentes Classe

Os **componentes de classe** são definidos como **classes ES6** que estendem **React.Component** e incluem um método **render()** que retorna elementos React. Os componentes de classe podem facilmente manter e gerenciar seu próprio estado interno usando

this.state. Ainda, os componentes de classe têm acesso a métodos de ciclo de vida (como **componentDidMount**, **componentDidUpdate**, **componentWillUnmount**) que permitem executar código em momentos específicos do ciclo de vida do componente.

Veja no quadro abaixo um exemplo de componente de classe com React:

```
import { Component } from 'react';

class Saudacao extends Component {
  render() {
    return <h1>Hello, {this.props.name}</h1>;
  }
}
```

O React chamará o método **render** sempre que precisar determinar o que deve ser exibido na tela. Geralmente, você retornará algum JSX a partir desse método, que descreve a UI que este componente deve renderizar. No exemplo acima, **<h1>Hello, {this.props.name}</h1>** é um elemento JSX que renderiza um cabeçalho de nível 1 (**<h1>**) com o texto "Hello" seguido pelo valor de **this.props.name**.

Observe que, assim como os componentes funcionais, um componente de classe pode receber informações do seu componente pai por meio de **props**. No entanto, a sintaxe para acessar **props** em um componente de classe é diferente. Por exemplo, se o componente pai renderizar **<Saudacao name="Taylor" />**, você pode acessar a propriedade **name** através de **this.props.name**.

Observação: Sempre inicie os nomes dos componentes em React com uma letra maiúscula. Isso é importante porque o React trata nomes de componentes começando com letras minúsculas como se fossem tags HTML nativas do DOM. Por exemplo, **<div />** é interpretado pelo React como uma tag HTML **div**, enquanto **<Welcome />** é reconhecido como um componente React. Porém, para que o React entenda **<Welcome />** como um componente, o nome **Welcome** precisa estar em escopo, ou seja, deve ser definido ou importado no arquivo onde está sendo utilizado.

3. Funcionamento de Props e Children no React

Em React, a passagem de dados entre componentes é uma das funcionalidades mais importantes, e isso é feito principalmente através de **props**. Além disso, o conceito de **children** é fundamental para criar componentes reutilizáveis e flexíveis, permitindo a composição de interfaces mais complexas.

Vamos detalhar como as **props** e **children** funcionam e como são utilizadas para compor componentes no React.

3.1 O Que São Props?

As **props** (abreviação de "propriedades") são o mecanismo pelo qual os componentes podem receber dados do componente pai. Eles são imutáveis, ou seja, um componente não pode modificar suas próprias **props**; ele apenas as usa para exibir conteúdo ou para passar esses dados adiante para outros componentes.

3.1.1 Como Funcionam as Props?

Quando você passa uma **prop** para um componente, ele a recebe como um argumento na função (para componentes funcionais) ou como um atributo **this.props** (para componentes de classe). Isso permite que os componentes sejam reutilizáveis e dinâmicos, já que eles podem exibir diferentes dados dependendo das props que recebem.

Veja no quadro a seguir um exemplo de uso de **props** em um componente funcional:

```
function Saudacao(props) {  
  return <h1>Olá, {props.nome}!</h1>;  
}  
function App() {  
  return (  
    <div>  
      <Saudacao nome="Maria" />  
      <Saudacao nome="João" />  
    </div>  
  );  
}  
export default App;
```

Nesse exemplo:

- **Saudacao** é um componente funcional que recebe uma **prop** chamada **nome**.
- No componente **App**, duas instâncias do componente **Saudacao** são criadas, passando valores diferentes ("Maria" e "João") como **props**.
- O componente **Saudacao** usa **{props.nome}** para exibir o nome correto.

No quadro abaixo, temos o mesmo exemplo com o componente **Saudacao**, mas utilizando o conceito de componente de classe:

```
import { Component } from 'react';
class Saudacao extends Component {
  render() {
    return <h1>Olá, {this.props.nome}!</h1>;
  }
}
class App extends Component {
  render() {
    return (
      <div>
        <Saudacao nome="Maria" />
        <Saudacao nome="João" />
      </div>
    );
  }
}
export default App;
```

Neste exemplo:

- O componente **Saudacao** agora é uma classe que estende **Component** e usa o método **render()** para retornar o JSX.
- A **props** agora é acessada através de **this.props.nome**, pois **this** é usado para referenciar as propriedades no contexto das classes.
- O componente **App** também foi convertido para uma classe com o método **render()** retornando o JSX, mantendo o comportamento original de renderizar múltiplas saudações.

O resultado no navegador, para ambos os casos, seria como ilustrado na figura abaixo:

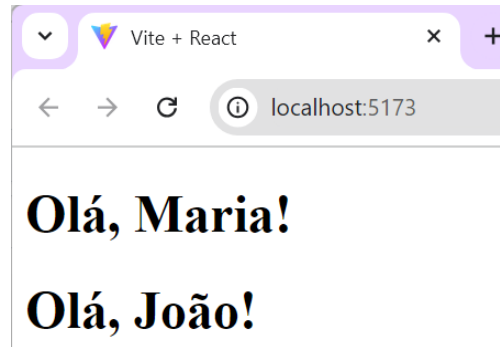


Figura 1: Exemplo de uso de props em componentes React

Descrição: A imagem mostra uma janela do navegador exibindo os textos "Olá, Maria!" e "Olá, João!" em negrito.

3.1.2 Imutabilidade das Props

Como as **props** são imutáveis, você não pode alterá-las diretamente dentro do componente. Se for necessário modificar o estado de um componente, você deve usar o estado local (via **useState**, por exemplo) ou passar funções através das props para que o componente filho possa disparar mudanças no componente pai.

3.2 Passagem de Funções via Props

As **props** não se limitam a dados estáticos. Você também pode **passar funções como props**, permitindo que o componente filho comunique-se com o componente pai ou execute ações a partir de eventos (como cliques de botão).

Veja no quadro abaixo um exemplo de passagem de funções com props:

```
function Botao(props) {  
  return <button onClick={props.onClick}>Clique aqui</button>;  
}  
  
function App() {  
  function mostrarMensagem() {  
    alert("Botão clicado!");  
  }  
}
```

```
return (  
  <div>  
    <Botao onClick={mostrarMensagem} />  
  </div>  
);  
}  
  
export default App;
```

Neste exemplo, o componente **Botao** recebe uma **prop** chamada **onClick**, que é uma função passada pelo componente **App**. Quando o botão é clicado, o React chama a função **mostrarMensagem** definida no componente pai.

Veja na figura abaixo o resultado da execução deste código no navegador.

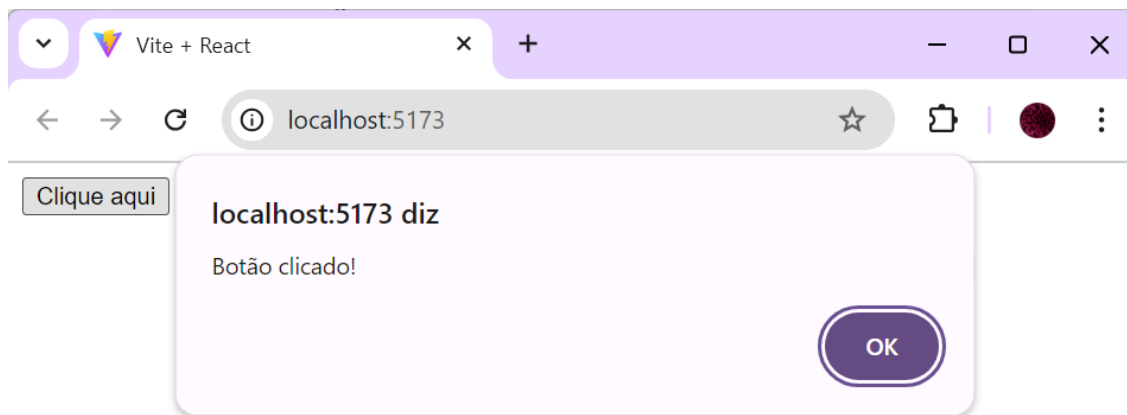


Figura 2: Exemplo de uso de passagem de funções via props em componentes React

Descrição: A imagem mostra uma janela do navegador exibindo uma aplicação web local rodando em "localhost:5173" com o título "Vite + React". Há um botão na página escrito "Clique aqui". Ao clicar no botão, uma janela de alerta aparece no centro da tela com a mensagem "Botão clicado!" e um botão "OK" para fechar o alerta.

3.3 Children: Composição de Componentes

Em React, a **prop** especial chamada **children** permite que você passe conteúdo dinâmico (como JSX ou outros componentes) dentro da marcação de um componente. Isso é essencial para a composição de componentes, já que você pode criar componentes genéricos e flexíveis que envolvem ou renderizam outros componentes dentro de si.

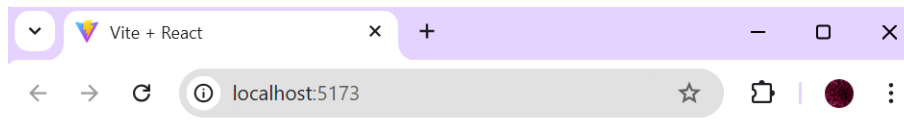
Veja no quadro abaixo um exemplo de uso de **children**:

```
function Caixa(props) {  
  return <div className="caixa">{props.children}</div>;  
}  
  
function App() {  
  return (  
    <div>  
      <Caixa>  
        <h1>Conteúdo 1</h1>  
      </Caixa>  
      <Caixa>  
        <h1>Conteúdo 2</h1>  
      </Caixa>  
    </div>  
  );  
}  
  
export default App;
```

Neste exemplo:

- O componente **Caixa** envolve outros componentes ou elementos em uma **<div>**.
- O conteúdo passado entre as tags de abertura e fechamento de **<Caixa>** (por exemplo, **<h1>Conteúdo 1</h1>**) é acessado pelo componente como **props.children**.
- O uso de **children** permite que o componente **Caixa** seja reutilizado, exibindo conteúdos diferentes em cada instância.

Veja na figura abaixo o resultado da execução deste código no navegador.



Conteúdo 1

Conteúdo 2

Figura 3: Exemplo de uso de children em componentes React

Descrição: A imagem mostra uma janela do navegador exibindo uma aplicação web local rodando em "localhost:5173" com o título "Vite + React". O conteúdo da página exibe dois textos grandes em negrito: "Conteúdo 1" e "Conteúdo 2", um abaixo do outro.

3.3.1 Composição Avançada com Children

Você pode criar componentes muito mais complexos utilizando a prop **children**, permitindo a composição de layouts e interfaces dinâmicas.

Veja no quadro abaixo um exemplo com de componente com múltiplos filhos:

```
function Layout(props) {  
  return (  
    <div>  
      <header>{props.header}</header>  
      <main>{props.children}</main>  
      <footer>{props.footer}</footer>  
    </div>  
  );  
}  
  
function App() {  
  return (  
    <Layout header={<h1>Cabeçalho</h1>} footer={<p>Rodapé</p>}>  
      <p>Conteúdo principal vai aqui!</p>  
    </Layout>  
  );  
}  
  
export default App;
```

Neste exemplo:

- O componente **Layout** organiza a página com um cabeçalho (**header**), um corpo (**children**) e um rodapé (**footer**).
- No componente **App**, o conteúdo do **header**, **footer** e **children** é passado dinamicamente.
- Isso exemplifica como **children** e outras **props** podem ser combinadas para criar componentes que envolvem outros elementos de maneira modular e flexível.

3.4 Diferenças entre Props e Children

Veja na tabela abaixo um resumo sobre o comparativo entre as diferenças sobre o uso de **props** e de **children**:

Aspecto	Props	Children
O que são	Dados ou funções passadas de um componente pai para um componente filho	Conteúdo aninhado passado entre as tags de abertura e fechamento de um componente
Sintaxe	Atributos em elementos JSX (<Componente prop={valor} />)	Conteúdo dentro do JSX (<Componente>conteúdo</Componente>)
Função principal	Configurar, parametrizar ou passar dados para o componente filho	Permitir a composição de outros componentes ou elementos dentro do componente
Exemplo	<Botao onClick={funcao} />	<Caixa> Texto dentro da caixa </Caixa>

3.5 Boas Práticas no uso de Props e Children

Veja nesta seção algumas sugestões de boas práticas no uso de **props** e **children**.

1. **Props Imutáveis:** Lembre-se de que as props devem sempre ser tratadas como imutáveis. Modificar uma prop diretamente dentro de um componente é uma má prática e pode causar erros inesperados.
2. **Desestruturação de Props:** Para manter o código limpo e legível, desestruture as props ao recebê-las nos componentes. Isso torna o código mais conciso e claro.

```
function Saudacao({ nome }) {  
  return <h1>Olá, {nome}</h1>;  
}
```

3. **Componentes Reutilizáveis:** Use **props** e **children** para criar componentes altamente reutilizáveis. Por exemplo, um componente de layout pode ser reutilizado com diferentes

conteúdos passados como **children**.

4. **Verificação de PropTypes:** Para melhorar a segurança e previsibilidade do código, você pode definir **PropTypes** para verificar o tipo das props esperadas, facilitando a depuração:

```
import PropTypes from "prop-types";

function Saudacao({ nome }) {
  return <h1>Olá, {nome}!</h1>;
}

Saudacao.propTypes = {
  nome: PropTypes.string.isRequired,
};
```

Observe que **PropTypes** é uma biblioteca usada no React para realizar a validação de tipos das **props** passadas para um componente. Isso permite que os desenvolvedores especifiquem quais tipos de dados seus componentes esperam receber, ajudando a detectar erros durante o desenvolvimento. Embora não impeça a execução do código em tempo de execução, o **PropTypes** emite avisos no console se uma **prop** inválida for passada, o que pode prevenir bugs e tornar o código mais robusto.

No exemplo anterior, em **Saudacao.propTypes = { nome: PropTypes.string.isRequired, },** estamos definindo os tipos das props que o componente **Saudacao** espera receber. Assim, a prop **nome** deve ser do tipo string e é obrigatória.

Se o componente **Saudacao** for usado corretamente, passando uma string para a prop **nome**, como no exemplo abaixo:

```
function App() {
  return <Saudacao nome="João" />;
}
```

O React não exibirá nenhum aviso, e o nome será exibido como esperado no navegador.

Porém, se um valor incorreto for passado, como um número:

```
function App() {  
  return <Saudacao nome={123} />;  
}
```

O React exibirá um aviso no console do Google DevTools, como ilustrado abaixo:



Figura 4: Exemplo de uso incorreto de PropTypes

Descrição: A imagem mostra uma janela do Google DevTools com uma mensagem de aviso (warning) em destaque. O aviso diz que a propriedade "nome" foi passada com o tipo "number", mas a aplicação esperava uma "string". O erro ocorre no componente "Saudacao", no arquivo "App.jsx", linha 12.

Ainda, se a prop **nome** não for passada:

```
function App() {  
  return <Saudacao/>;  
}
```

O React exibirá outro aviso no console do Google DevTools, como ilustrado abaixo:



Figura 5: Exemplo de uso incorreto de PropTypes

Descrição: A imagem mostra uma janela do Google DevTools com uma mensagem de aviso (warning) em destaque. O aviso informa que a propriedade "nome" foi marcada como obrigatória no componente "Saudacao", mas seu valor está como "undefined" (não definido). O erro está no arquivo "App.jsx", linha 12.

4. Renderização Condicional e Renderização de Listas em React

A **renderização condicional** e a **renderização de listas** são dois conceitos essenciais no React para exibir conteúdo dinâmico, ou seja, ajustar a interface com base em dados variáveis ou condições específicas. Ambos permitem que os desenvolvedores criem interfaces interativas e dinâmicas, com elementos que aparecem ou desaparecem de acordo com o **estado** ou **props**, e listas de itens que podem ser geradas a partir de arrays.

4.1 Renderização Condicional

A renderização condicional permite que o React exiba diferentes componentes ou elementos de acordo com uma condição lógica. É semelhante às instruções condicionais (como **if**, **else**, **switch**) usadas no JavaScript. Existem várias maneiras de fazer isso em React:

- Usando **if-else**;
- Usando operador ternário;
- Usando operador lógico **&&**.

4.1.1 Usando if-else

Você pode usar a estrutura de controle **if-else** para condicionar a renderização de diferentes elementos. Veja um exemplo no quadro abaixo:

```
function Saudacao({ isLogado }) {  
  if (isLogado) {  
    return <h1>Bem-vindo de volta!</h1>;  
  } else {  
    return <h1>Por favor, faça login.</h1>;  
  }  
}
```

Nesse exemplo:

- O componente **Saudacao** recebe uma prop **isLogado**.
- Se **isLogado** for **true**, ele renderiza a mensagem "Bem-vindo de volta!".
- Se for **false**, renderiza "Por favor, faça login."

4.1.2 Usando operador ternário

O operador ternário é uma maneira concisa de realizar renderização condicional em React. Ele permite que você escreva uma condição em uma única linha, retornando um valor ou um JSX específico com base em se a condição é verdadeira ou falsa.

A sintaxe do operador ternário é:

```
condição ? expressão1 : expressão2;
```

Onde:

- **condição:** A condição que será avaliada (por exemplo, uma comparação ou variável booleana).
- **expressão1:** O que será retornado/renderizado se a condição for verdadeira.
- **expressão2:** O que será retornado/renderizado se a condição for falsa.

Veja no quadro abaixo um exemplo básico de como o operador ternário funciona:

```
function Saudacao({ isLogado }) {  
  return <h1>{isLogado ? "Bem-vindo de volta!" : "Por favor, faça login."}</h1>;  
}
```

Neste exemplo:

- A prop **isLogado** (um valor booleano) determina qual mensagem será exibida.
- Se **isLogado** for **true**, a expressão retorna **<h1>Bem-vindo de volta!</h1>**.
- Se **isLogado** for **false**, a expressão retorna **<h1>Por favor, faça login.</h1>**.

Neste exemplo, **isLogado ? 'Bem-vindo de volta!' : 'Por favor, faça login.'** é uma expressão que alterna o texto com base no valor de **isLogado**.

O operador ternário pode ser usado não apenas para renderizar diferentes textos, mas também diferentes componentes. Vamos ver no quadro abaixo um exemplo onde alternamos entre dois componentes com base em uma condição:

```
function UsuarioLogado() {  
  return <h1>Você está logado!</h1>;  
}
```

```
function UsuarioDeslogado() {  
  return <h1>Você não está logado!</h1>;  
}  
function Saudacao({ isLogado }) {  
  return (  
    <div>  
      {isLogado ? <UsuarioLogado /> : <UsuarioDeslogado />}  
    </div>  
  );  
}
```

Neste exemplo:

- Se **isLogado** for **true**, o componente **UsuarioLogado** será renderizado.
- Se **isLogado** for **false**, o componente **UsuarioDeslogado** será exibido.

4.1.3 Usando operador lógico &&

Em React, o operador lógico **&&** (AND) é frequentemente utilizado para realizar renderização condicional de forma concisa. Ele é uma alternativa ao uso do operador ternário quando você quer renderizar algo apenas se uma condição for verdadeira, sem a necessidade de um **"else"** (ou seja, sem precisar lidar com o que acontece quando a condição é falsa).

A sintaxe do operador && é:

```
condição && elemento;
```

Onde:

- Se a **condição** for **true**, o elemento será renderizado.
- Se a **condição** for **false**, o React não renderizará nada (pois false ou null não são renderizados no DOM).

No quadro abaixo está um exemplo simples que demonstra o uso do operador **&&** para renderizar um componente condicionalmente:

```
function Saudacao({ isLogado }) {  
  return (  
    <div>  
      {isLogado && <h1>Bem-vindo de volta!</h1>}  
    </div>  
  );  
}
```

Nesse exemplo:

- Se **isLogado** for **true**, a mensagem "**Bem-vindo de volta!**" será exibida.
- Se **isLogado** for **false**, o React não renderizará o `<h1>Bem-vindo de volta!</h1>`.

4.2 Renderização de Listas

A renderização de listas no React envolve exibir múltiplos elementos com base em uma coleção de dados, geralmente um array. O método mais comum para renderizar listas é usando o método **JavaScript .map()**, que itera sobre um array e retorna um novo array de elementos JSX.

A sintaxe básica de **.map()** é a seguinte:

```
array.map((elemento, index) => {  
  // Retorna um valor (ou JSX no caso de React)  
});
```

Onde:

- **elemento**: O valor atual do array durante a iteração.
- **index**: O índice do elemento no array (opcional, mas muitas vezes útil em React para gerar chaves únicas).
- **retorno**: O valor transformado que será adicionado ao novo array.

Quando você renderiza uma lista, o React recomenda o uso de **chaves** (ou **keys**) para identificar de forma única cada elemento da lista. Isso ajuda o React a identificar quais itens mudaram, foram adicionados ou removidos, e otimiza a atualização do DOM.

Veja no quadro abaixo um exemplo simples de renderização de lista no React:

```
function ListaNomes() {  
  const nomes = ['Maria', 'João', 'Pedro'];  
  return (  
    <ul>  
      {nomes.map((nome, index) => (  
        <li key={index}>{nome}</li>  
      ))}  
    </ul>  
  );  
}
```

Neste exemplo:

- A variável **nomes** é um array contendo strings.
- O método **.map()** percorre o array e retorna um elemento **** para cada nome.
- A **key** é uma **prop** obrigatória para ajudar o React a identificar de forma eficiente os itens da lista. Nesse exemplo, usamos o **index** como chave, embora em muitos casos seja melhor usar um identificador único quando disponível.

Veja abaixo o resultado da renderização do componente **ListaNomes** no navegador:

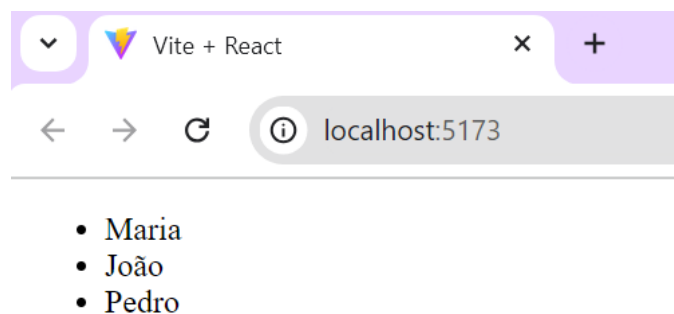


Figura 6: Exemplo de uso de renderização de lista em React

Descrição: A imagem mostra uma janela do navegador exibindo uma aplicação web local rodando em "localhost:5173" com o título "Vite + React". O conteúdo da página exibe uma lista com três nomes: "Maria", "João" e "Pedro", cada um precedido por um ponto de lista (marcador).

Se os itens da lista forem objetos, você pode acessar suas propriedades dentro do método **map()**, conforme quadro abaixo:

```
function ListaUsuarios() {  
  const usuarios = [  
    { id: 1, nome: "Maria" },  
    { id: 2, nome: "João" },  
    { id: 3, nome: "Pedro" },  
  ];  
  return (  
    <ul>  
      {usuarios.map((usuario) => (  
        <li key={usuario.id}>{usuario.nome}</li>  
      ))}  
    </ul>  
  );  
}
```

Neste exemplo:

- Cada item do array **usuarios** é um objeto com **id** e **nome**.
- O **.map()** percorre o array e retorna um novo array de **** com o nome do usuário.
- Usamos **usuario.id** como **key**, pois é um **identificador único**.

O **.map()** não apenas transforma dados em JSX, mas também pode ser combinado com lógica condicional e outras manipulações de array. Por exemplo, você pode filtrar os itens antes de renderizá-los. Abaixo está um exemplo em que apenas usuários ativos são exibidos:

```
function ListaUsuarios({ usuarios }) {  
  return (  
    <ul>  
      {usuarios  
        .filter((usuario) => usuario.ativo)  
        .map((usuario) => (  
          <li key={usuario.id}>{usuario.nome}</li>  
        ))}  
    </ul>  
  );  
}
```

```
function App() {  
  const usuarios = [  
    { id: 1, nome: "Maria", ativo: true },  
    { id: 2, nome: "João", ativo: false },  
    { id: 3, nome: "Pedro", ativo: true },  
  ];  
  return <ListaUsuarios usuarios={usuarios} />;  
}
```

Neste exemplo:

- Primeiro, usamos **.filter()** para retornar apenas os usuários que estão **ativo: true**.
- Em seguida, aplicamos **.map()** para renderizar apenas os usuários filtrados.

Veja abaixo o resultado da renderização do componente **ListaUsuarios** no navegador:

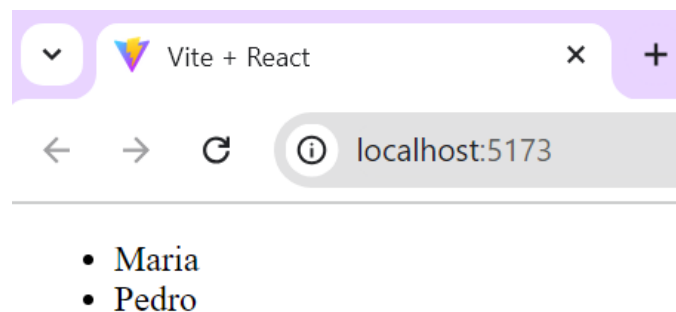


Figura 7: Exemplo de uso de renderização de lista com o uso de método filter

Descrição: A imagem mostra uma janela do navegador exibindo uma aplicação web local rodando em "localhost:5173" com o título "Vite + React". O conteúdo da página exibe uma lista com dois nomes: "Maria" e "Pedro", cada um precedido por um ponto de lista (marcador).

4.3 Combinação de Renderização Condicional e Listas

A renderização condicional de listas permite exibir uma lista apenas se uma condição específica for atendida. Um exemplo clássico é exibir uma lista de itens se ela contiver dados ou exibir uma mensagem quando a lista estiver vazia.

Veja um exemplo no quadro abaixo:

```
function ListaNomes({ nomes }) {  
  return (  
    <div>  
      {nomes.length > 0 ? (  
        <ul>  
          {nomes.map((nome, index) => (  
            <li key={index}>{nome}</li>  
          ))}  
        </ul>  
      ) : (  
        <p>Nenhum nome disponível</p>  
      )}  
    </div>  
  );  
}  
  
function App() {  
  const nomes = ["Maria", "João", "Pedro"];  
  return <ListaNomes nomes={nomes} />;  
}  
  
export default App;
```

Neste exemplo:

- A condição **nomes.length > 0** verifica se a lista de nomes contém algum item.
- Se a condição for **verdadeira**, o array **nomes** é mapeado para renderizar uma lista de **** com os nomes.
- Se a lista estiver **vazia** (ou seja, **nomes.length === 0**), uma mensagem **"Nenhum nome disponível"** é exibida.

Referências

O material desenvolvido para este capítulo baseou-se nas seguintes obras:

- DEVLIN BASILAN DULDULAO, D. B.; CABAGNOT, R. J. L. Practical Enterprise React: Become an Effective React Developer in Your Team. USA: APRESS, 2021.
- ELROM, E. React and Libraries: Your Complete Guide to the React Ecosystem. USA: APRESS, 2021.
- MDN WEB DOCS. Getting started with React. Disponível em: https://developer.mozilla.org/en-US/docs/Learn/Tools_and_testing/Client-side_JavaScript_frameworks/React_getting_started. Acessado em 21 jun. 2024.
- REACT DOCS. Disponível em <https://pt-br.react.dev/>. Acessado em 15 jun. 2024.
- ROLDÁN, C. S. React 17: Design Patterns and Best Practices. UK: Packt Publishing, 2021.
- SCOTT, A. D. JavaScript Everywhere. USA: O'Reilly Media, 2020.
- UZAYR, S. B.; CLOUD, N.; AMBLER, T. JavaScript Frameworks for Modern Web Development: The Essential Frameworks, Libraries, and Tools to Learn Right Now. USA: APRESS, 2019.
- W3SCHOOLS. React Tutorial. Disponível em: <https://www.w3schools.com/react>. Acessado em 17 jun. 2024.